

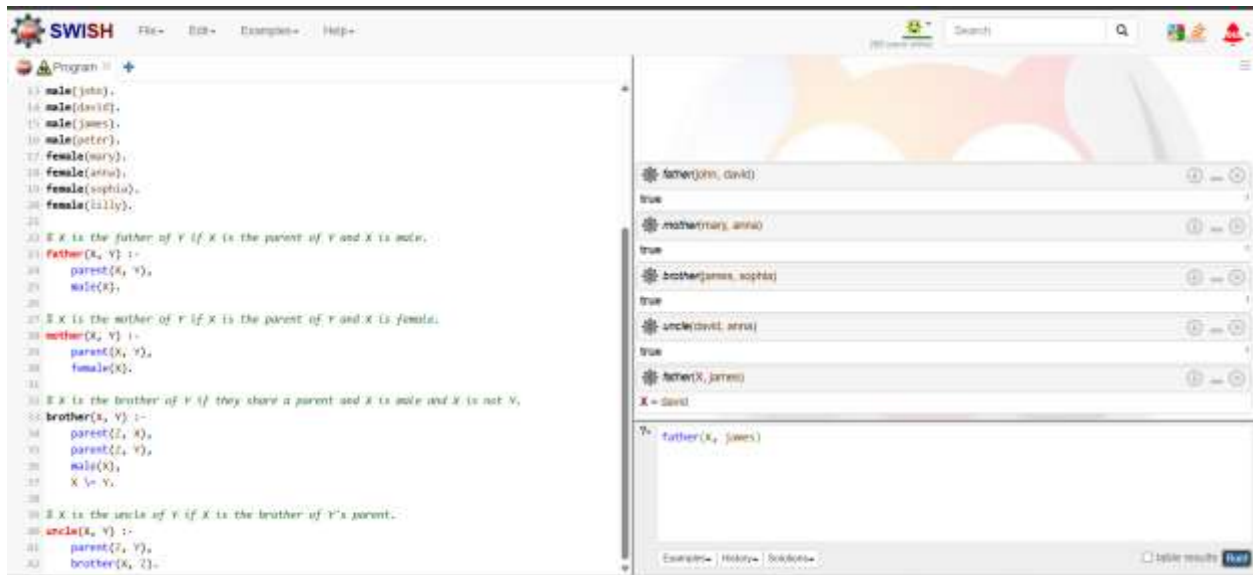
Department of Computer Engineering  
University of Peradeniya  
CO523 - Programming Languages  
Lab 10: Logical Programming Paradigm

E/20/449

WIJewardhana A.R.S.S.

## 5. Lab Tasks

### 5.1. Building a Family Knowledge Base



The objective of this task is to extend a family tree to three generations and define relational rules using the logical programming paradigm.

#### I. Knowledge Base (Facts)

In Prolog, facts are statements describing relationships that are always true. We define the lineage using parent/2 and identify individuals using male/1 and female/1.

#### II. Logical Rules

Rules are logical implications where a conclusion is reached if specific conditions are met.

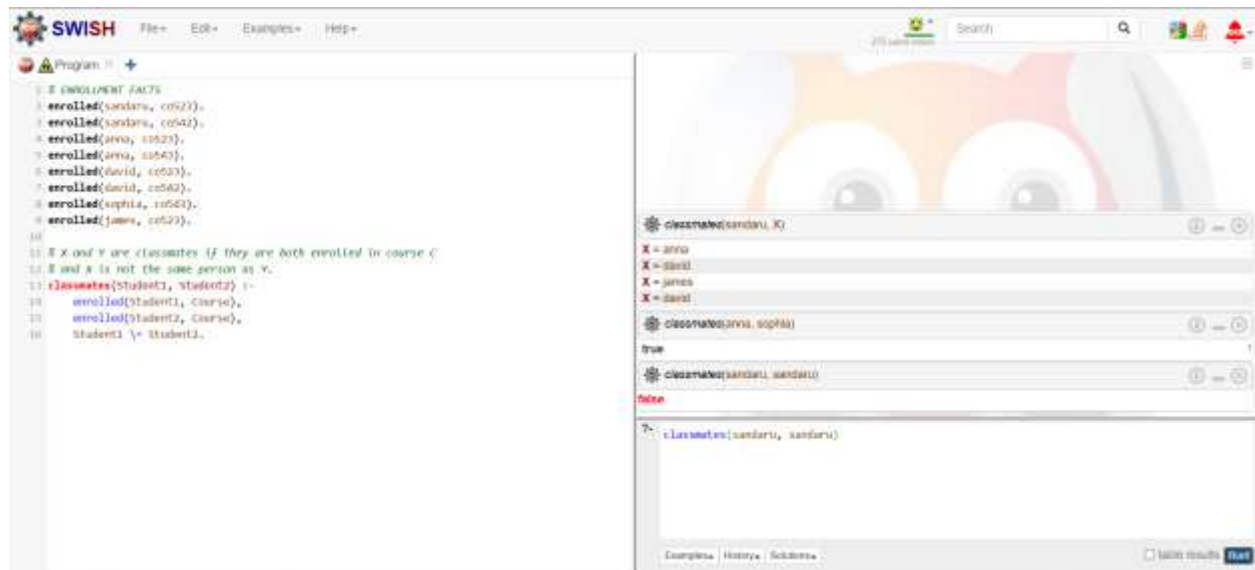
| Relationship | Prolog Rule Implementation             | Logic Explanation                                         |
|--------------|----------------------------------------|-----------------------------------------------------------|
| Father       | father(X, Y) :- parent(X, Y), male(X). | X is the father of Y if X is a parent of Y AND X is male. |

|         |                                                                            |                                                                         |
|---------|----------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Mother  | <code>mother(X, Y) :- parent(X, Y), female(X).</code>                      | X is the mother of Y if X is a parent of Y AND X is female.             |
| Brother | <code>brother(X, Y) :- parent(Z, X), parent(Z, Y), male(X), X \= Y.</code> | X and Y share a parent Z, X is male, and X is not the same person as Y. |
| Uncle   | <code>uncle(X, Y) :- parent(Z, Y), brother(X, Z).</code>                   | X is an uncle of Y if X is the brother of Y's parent (Z).               |

### III. Query Execution & Results

| Query                                   | Explanation of Output                                                                                 |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>?- father(john, david).</code>    | Returns true because John is a parent of David and is male.                                           |
| <code>?- mother(mary, anna).</code>     | Returns true because Mary is a parent of Anna and is female.                                          |
| <code>?- brother(james, sophia).</code> | Returns true because they both have the parent David, and James is male.                              |
| <code>?- uncle(david, anna).</code>     | Returns true because David is the brother of Mary, and Mary is the parent of Anna.                    |
| <code>?- father(X, james).</code>       | Returns <code>X = david</code> because Prolog finds the value that satisfies the father relationship. |

## 5.2. Unification and Constraints



This task focuses on using the unification process to identify relationships within an educational database. It will learn to use the inequality operator `\=` to apply constraints, ensuring that logical results remain valid (Example: a student cannot be their own classmate).

### 1. Define Student Enrollment Facts

First, we must create a database of students and the courses they are taking. Use the format `enrolled(student_name, course_code)`.

```
% ENROLLMENT FACTS
```

```
enrolled(sandaru, co523).
```

```
enrolled(sandaru, co542).
```

```
enrolled(anna, co523).
```

```
enrolled(anna, co543).
```

```
enrolled(david, co523).
```

```
enrolled(david, co542).
```

```
enrolled(sophia, co543).
```

```
enrolled(james, co523).
```

### 2. Define the Classmates Rule

A classmate is defined as a student who is enrolled in the same course as another student.

Crucially, we must use the `\=` operator to ensure the system doesn't list a student as a classmate of themselves.

```
% X and Y are classmates if they are both enrolled in course C
```

```
% and X is not the same person as Y.
```

```
classmates(Student1, Student2) :-
```

```
    enrolled(Student1, Course),
```

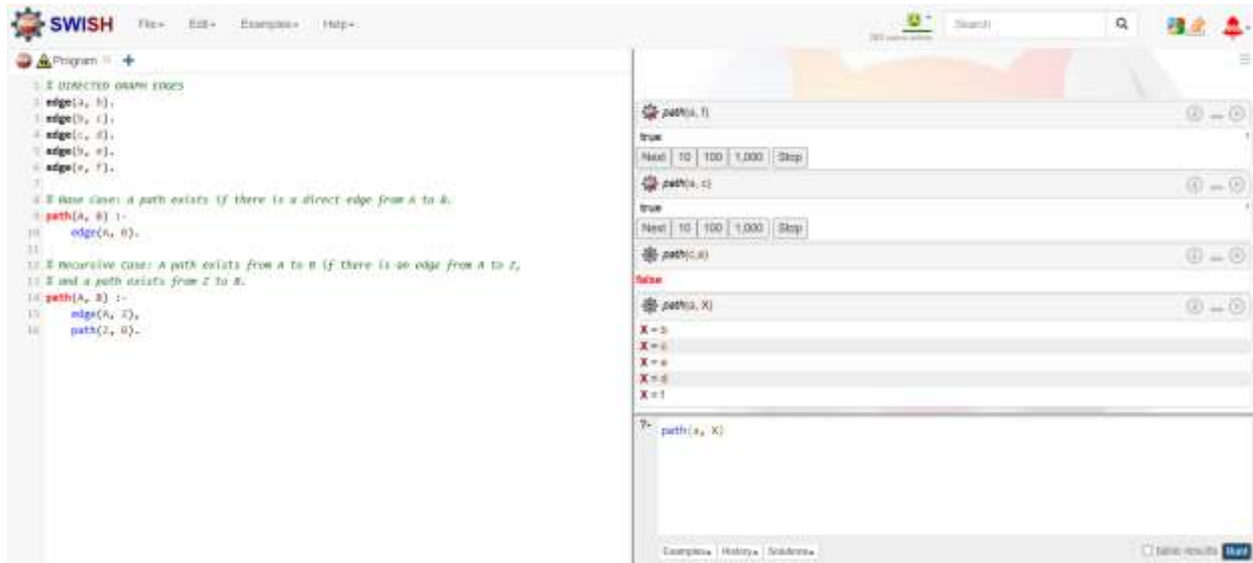
```
    enrolled(Student2, Course),
```

```
    Student1 \= Student2.
```

### 3. Run Queries and Analyze Results

| Query                            | Expected Output                           | Logic Explanation                                                       |
|----------------------------------|-------------------------------------------|-------------------------------------------------------------------------|
| ?- classmates(sandaru, X).       | X = anna, X = david, X = james (and more) | Prolog finds all students X who share at least one Course with sandaru. |
| ?- classmates(anna, sophia).     | true                                      | Both are enrolled in co543, satisfying the rule.                        |
| ?- classmates(sandaru, sandaru). | false                                     | The constraint Student1 \= Student2 prevents this from being true.      |

## 5.3. Implementing Recursive Logic



This involves modeling a directed graph and defining a recursive rule to determine if a path exists between two nodes. Recursion is a fundamental concept in Prolog, where a rule calls itself to solve a problem by breaking it down into smaller steps.

### 1. Define the Graph (Facts)

In a directed graph, an edge is a one-way connection from one node to another.

% DIRECTED GRAPH EDGES

edge(a, b).

edge(b, c).

edge(c, d).

edge(b, e).

edge(e, f).

### Step 2: Define the Recursive Path Rule

The path(A, B) rule consists of two parts: a base case and a recursive case.

% Base Case: A path exists if there is a direct edge from A to B.

path(A, B) :-

edge(A, B).

% Recursive Case: A path exists from A to B if there is an edge from A to Z,

% and a path exists from Z to B.

path(A, B) :-

edge(A, Z),

path(Z, B).

### 3. Run Queries and Analyze Results

| Query          | Result           | Logic Explanation                                                                              |
|----------------|------------------|------------------------------------------------------------------------------------------------|
| ?- path(a, c). | true             | Recursive Step: edge(a, b) -> edge(b, c). Prolog finds the connection through node 'b'.        |
| ?- path(a, f). | true             | Multi-step: edge(a, b) -> edge(b, e) -> edge(e, f). Prolog uses recursion to follow the chain. |
| ?- path(c, a). | false            | Since the edges are directed, there is no way to go backward from 'c' to 'a'.                  |
| ?- path(a, X). | X=b, X=c, X=d... | Using Backtracking, Prolog finds every node reachable from 'a'.                                |